What we're doing

How well do ESMFold (`facebook/esmfold_v1`) and ESMFold2 (`biohub/ESMFold2`) predict pyconfind-defined contacts on our eval set, compared to Protenix v2 (single-sequence and MSA)? We extend exp74 with two more single-sequence structure predictors, scored the same way: run pyconfind on the predicted structure and rank candidate pairs by contact degree, against one shared pyconfind ground truth.

Method

Same 554-protein eval set as exp74 (FoldBench-100 + 454 exp65 low-MSA / novel-fold candidates), same pyconfind ground truth (degree ≥ 0.001, sep ≥ 6), same metric (contacts @ L / L/2 / L/5 and R-precision; aggregate + short / medium / long). ESMFold runs once per protein (deterministic, num_recycles=4). ESMFold2 runs single-sequence, best-of-N diffusion samples (num_loops=20, num_sampling_steps=100), keeping the top-1 by confidence. Both on Modal (H100); all predicted structures saved to the HF bucket for re-scoring. Protenix v2 numbers are reused from exp74.

Results

All 552 proteins folded by both models (0 failures); pyconfind GT alignment ≥ 0.95 everywhere. R-precision (ceiling 1.0/protein), structure predictor:

FoldBench-100 — Protenix·SS 0.28, Protenix·MSA 0.85, ESMFold 0.76, ESMFold2 0.81. exp65 (low-MSA) — Protenix·SS 0.67, Protenix·MSA 0.80, ESMFold 0.76, ESMFold2 0.78.

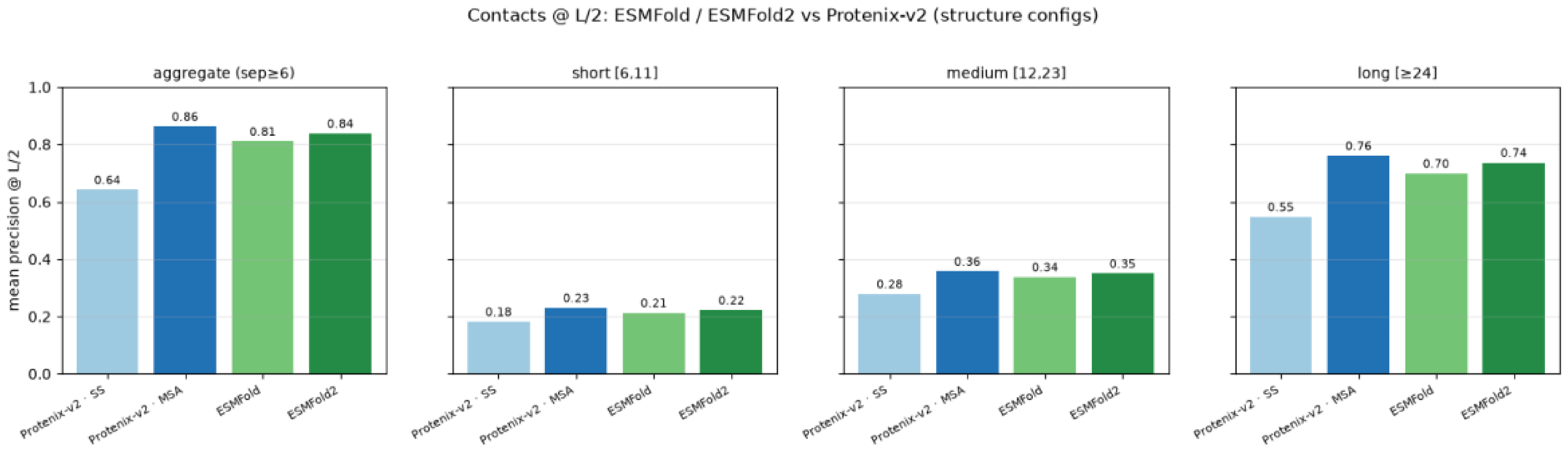
ESMFold2 is the best single-sequence contact predictor: single-seq ESMFold2 ≈ MSA-mode Protenix on natural proteins, and far above single-seq Protenix.

Takeaway

In the low-MSA / novel-fold regime the single-seq vs MSA gap nearly vanishes: single-sequence ESMFold2 lands within ~0.02 of MSA-mode Protenix on exp65, and for orphan proteins (Neff≈1) it's on par with or above MSA Protenix. A strong single-sequence folder loses little where MSAs are shallow — the regime that matters for MarinFold. ESMFold ~1.5 s/protein, ESMFold2 ~21 s/protein (best-of-5).

contacts_at_L_2_by_model_and_range.png

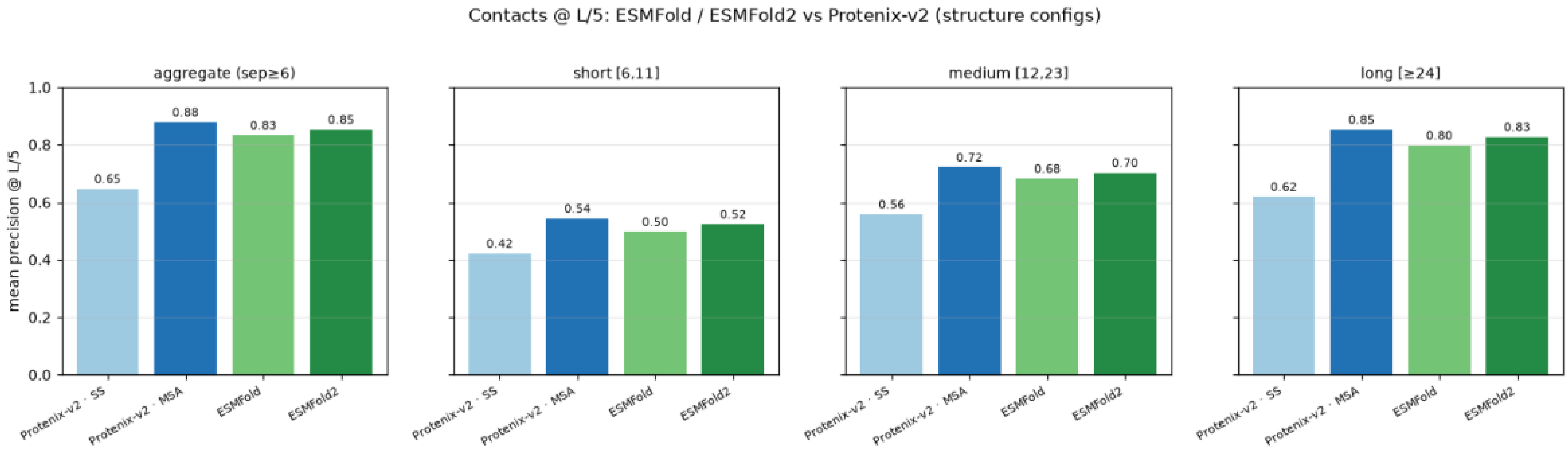
Mean precision @ L/2 vs pyconfind contacts (degree≥0.001, sep≥6), per model, aggregate and by range. All configs rank candidate pairs by pyconfind contact degree on the predicted structure.




```
$ plot.py --precision-csv data/contact_precision_all.csv --out plots
```

contacts_at_L_5_by_model_and_range.png

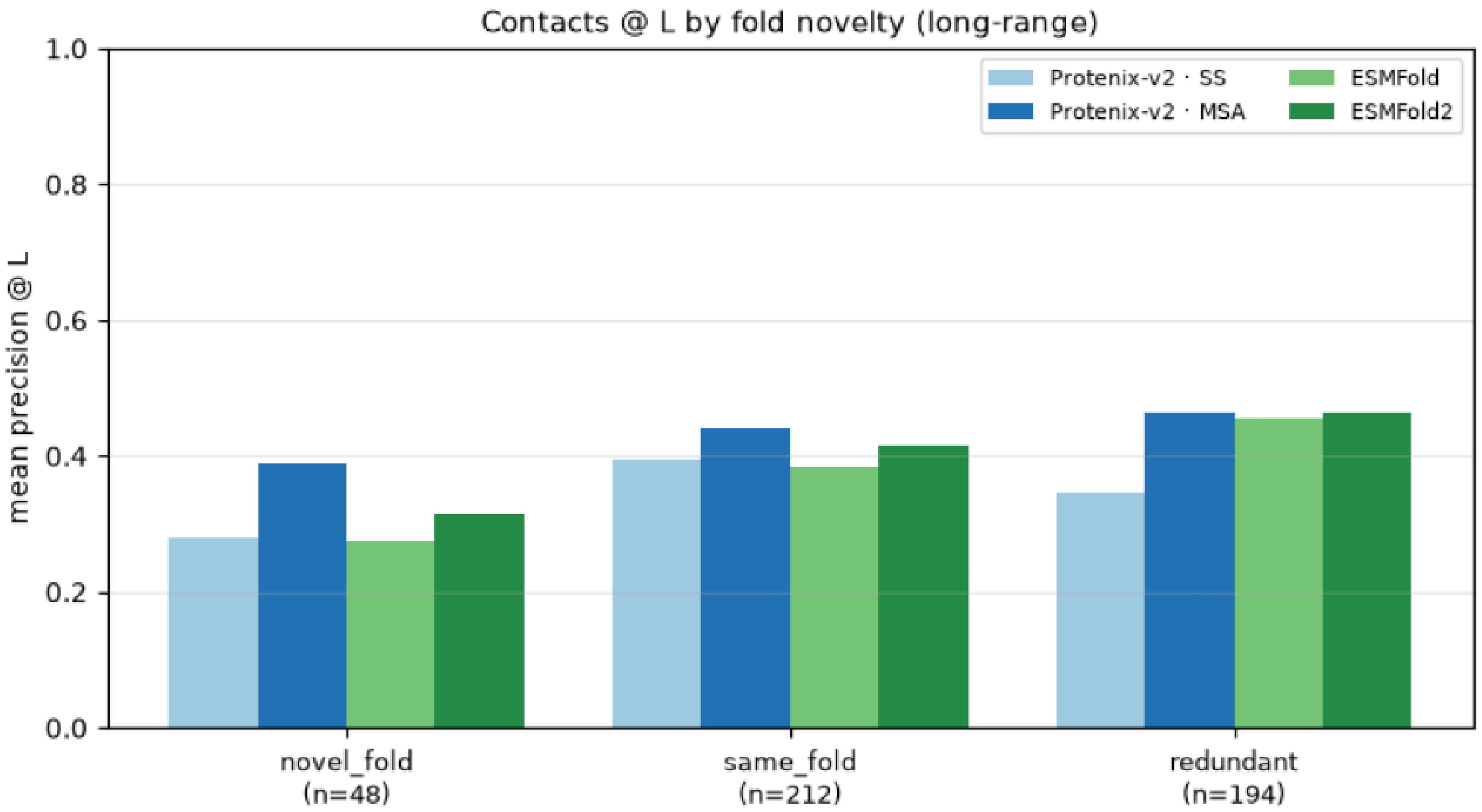
Mean precision @ L/5 vs pyconfind contacts (degree≥0.001, sep≥6), per model, aggregate and by range. All configs rank candidate pairs by pyconfind contact degree on the predicted structure.




```
$ plot.py --precision-csv data/contact_precision_all.csv --out plots
```

contacts_at_L_by_fold_verdict.png

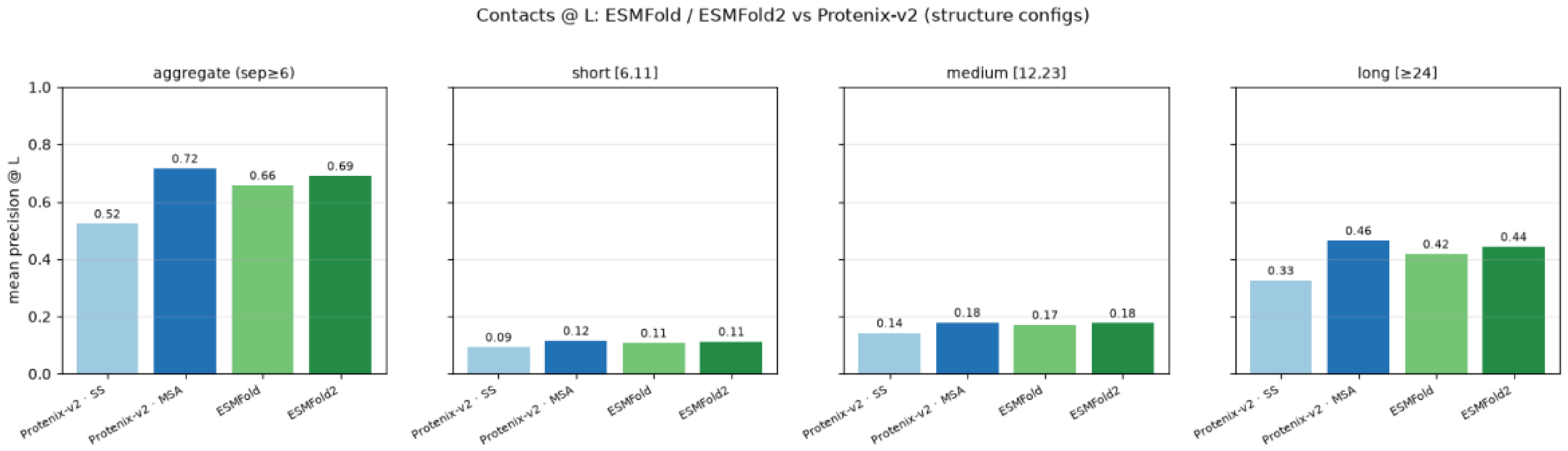
Contacts @ L by fold novelty (long-range)



`$ plot.py --precision-csv data/contact_precision_all.csv --out plots`

contacts_at_L_by_model_and_range.png

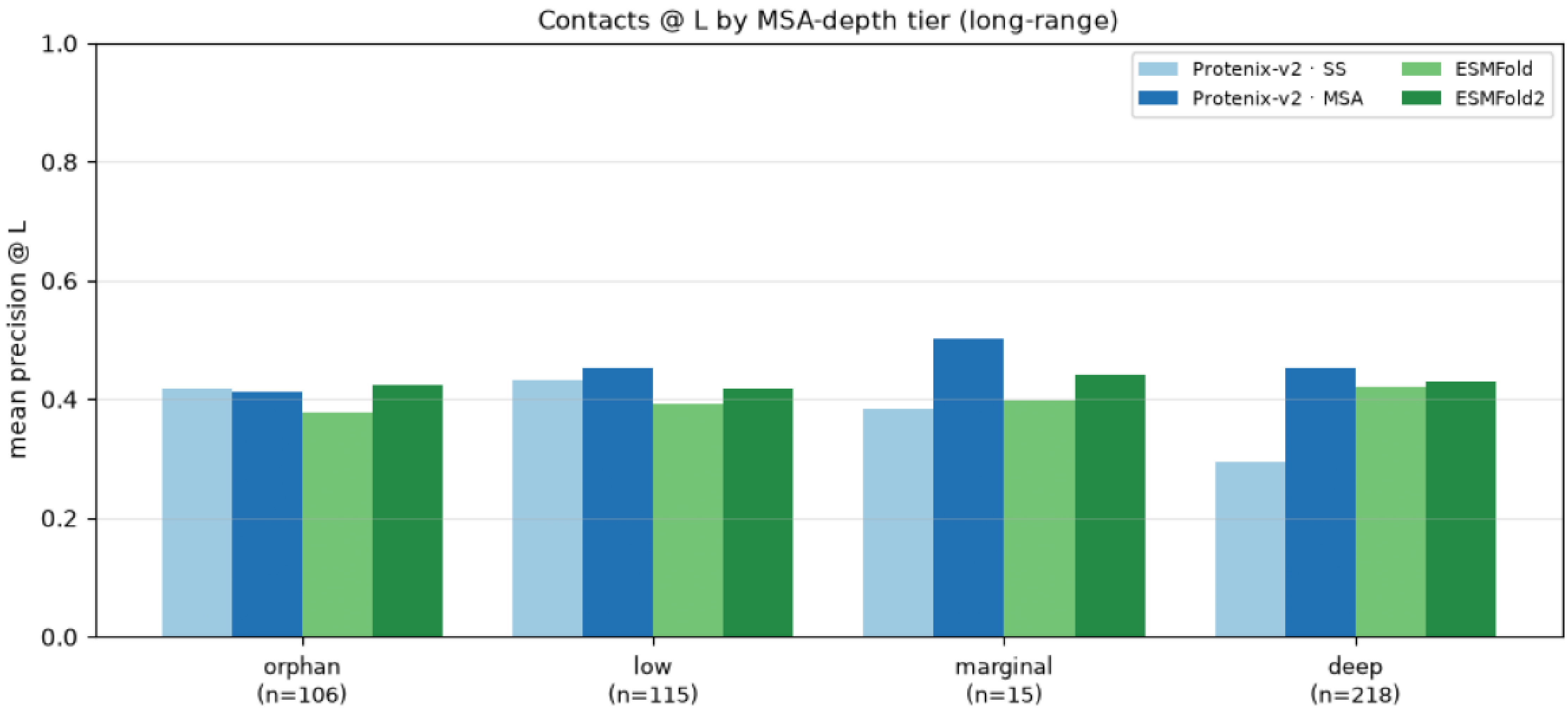
Mean precision @ L vs pyconfind contacts (degree≥0.001, sep≥6), per model, aggregate and by range. All configs rank candidate pairs by pyconfind contact degree on the predicted structure.




```
$ plot.py --precision-csv data/contact_precision_all.csv --out plots
```

contacts_at_L_by_neff_tier.png

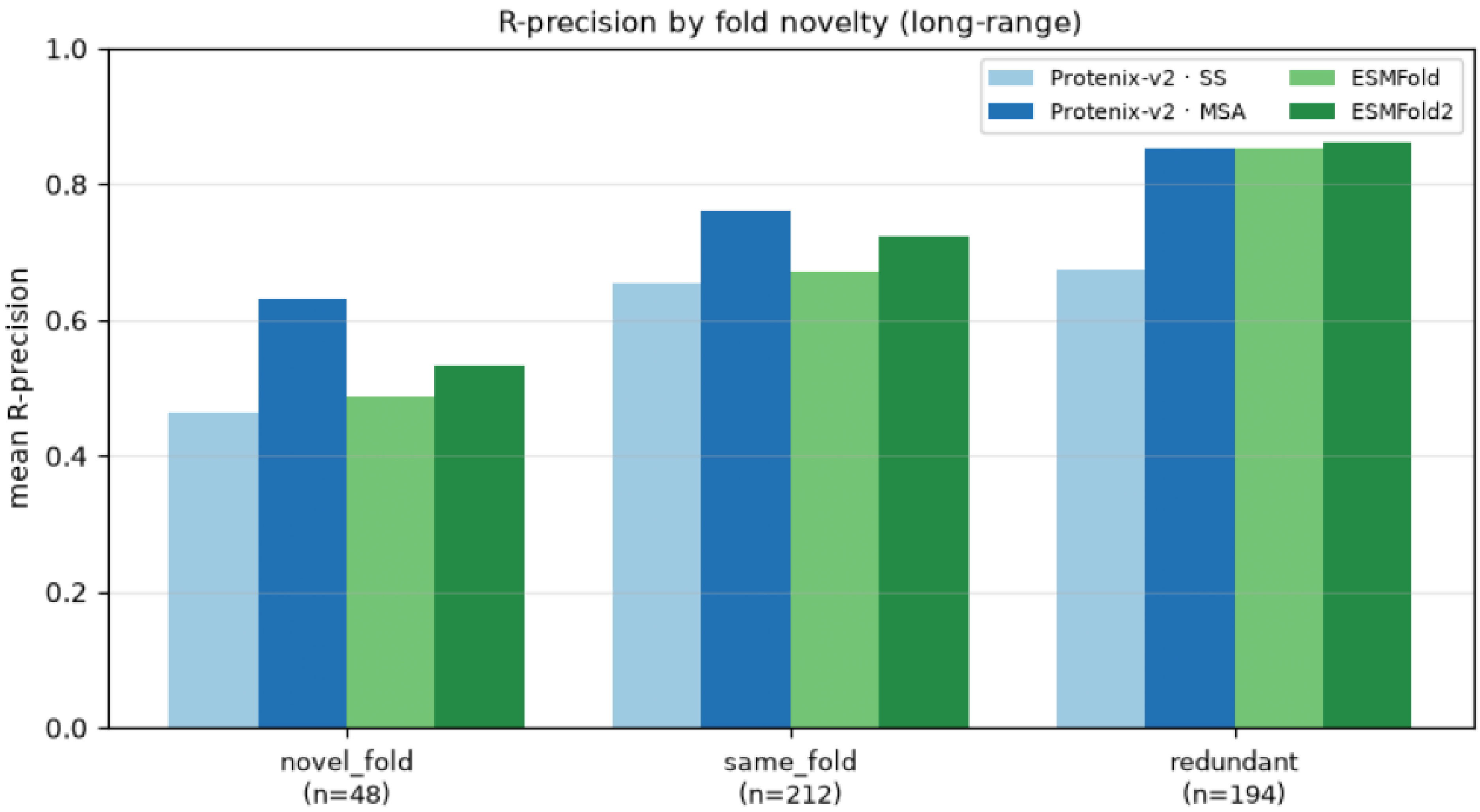
Contacts @ L by MSA-depth tier (long-range)




```
$ plot.py --precision-csv data/contact_precision_all.csv --out plots
```

contacts_at_R_by_fold_verdict.png

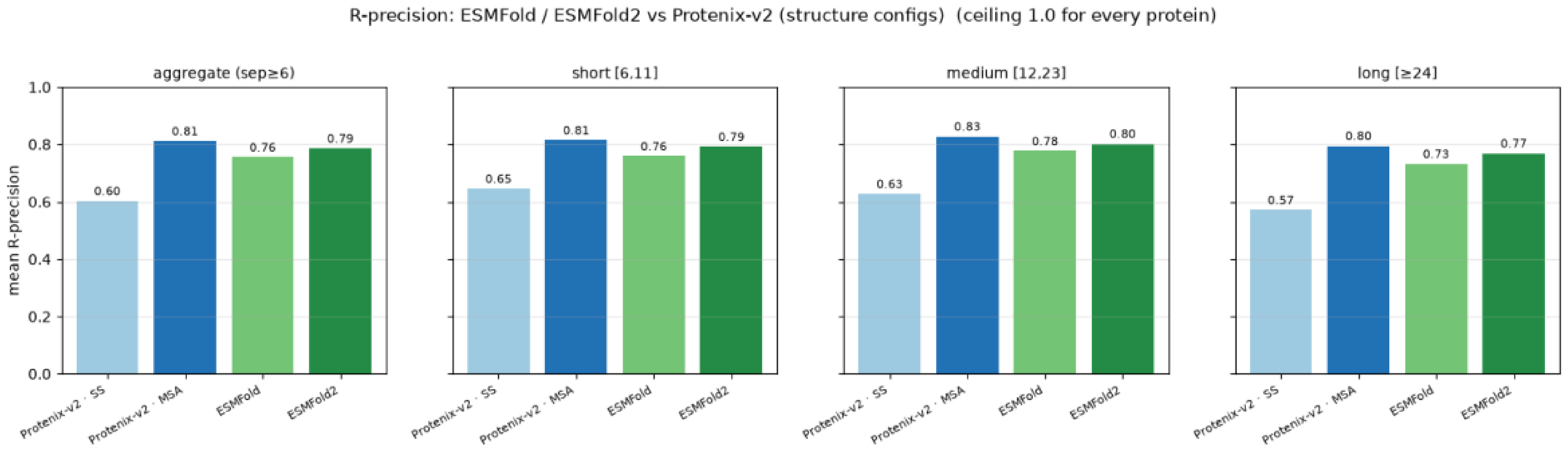
R-precision by fold novelty (long-range)



`$ plot.py --precision-csv data/contact_precision_all.csv --out plots`

contacts_at_R_by_model_and_range.png

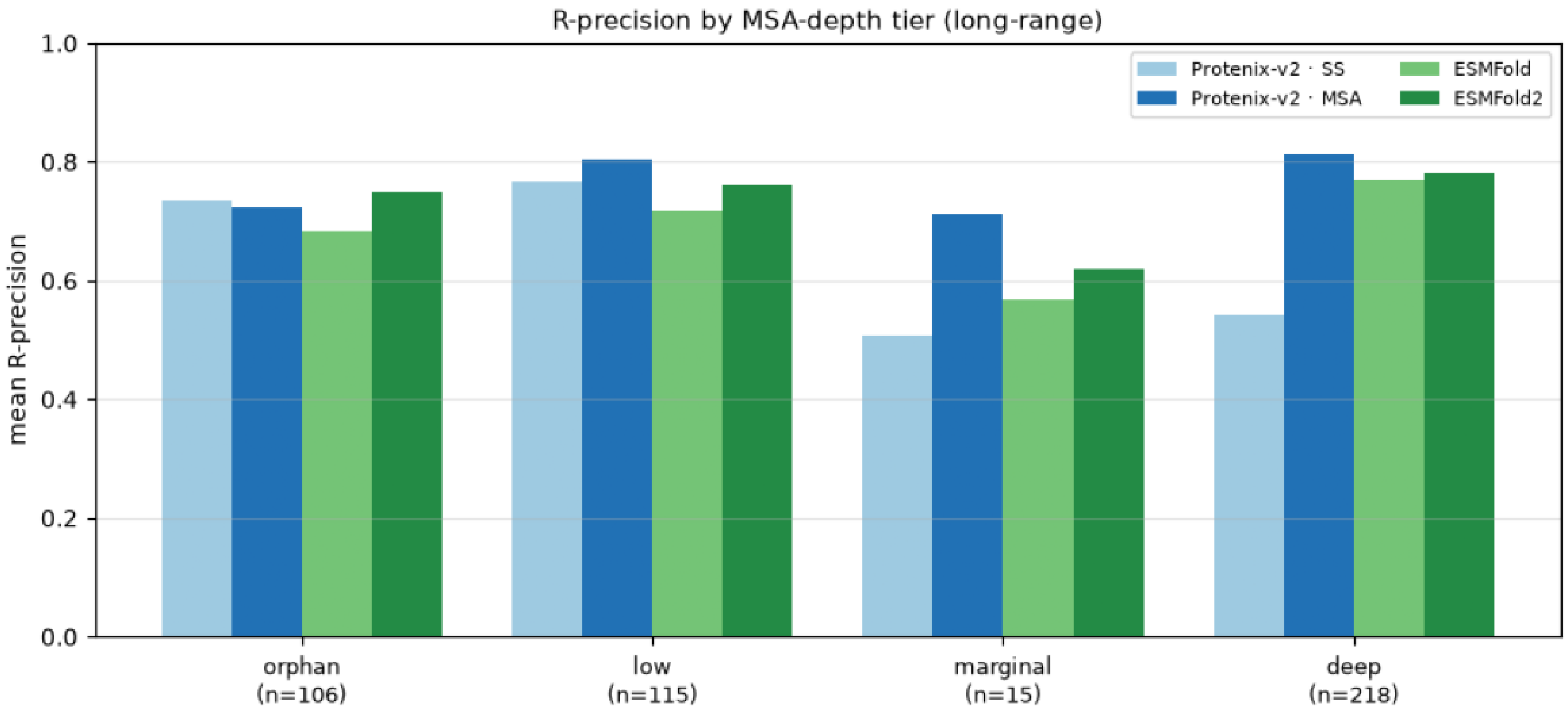
Mean R-precision vs pyconfind contacts (degree≥0.001, sep≥6), per model, aggregate and by range. All configs rank candidate pairs by pyconfind contact degree on the predicted structure. R-precision cutoff = #true contacts, so a perfect ranker = 1.0.




```
$ plot.py --precision-csv data/contact_precision_all.csv --out plots
```

contacts_at_R_by_neff_tier.png

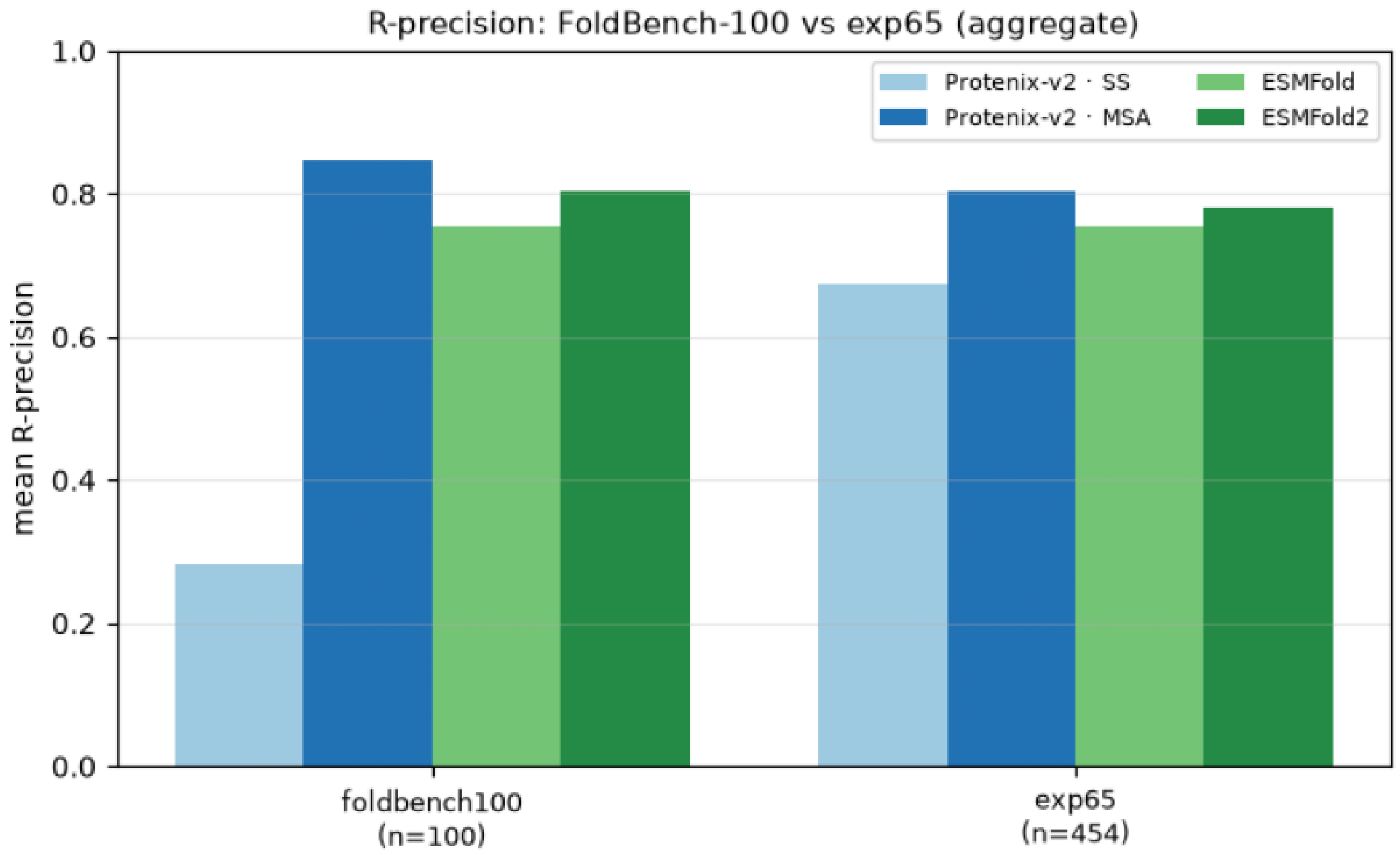
R-precision by MSA-depth tier (long-range)



$ plot.py --precision-csv data/contact_precision_all.csv --out plots

contacts_at_R_foldbench_vs_exp65.png

R-precision: FoldBench-100 vs exp65 (aggregate)




```
$ plot.py --precision-csv data/contact_precision_all.csv --out plots
```